What to eat? Build Brief

Oct 13, 2026 · @Brenten Spikes

Overview

"What to eat?" is a mobile app that ends the nightly dinner debate by giving one fun, tappable answer in seconds, whether tonight means cooking, fast food, dining out, or delivery.

- **Who it's for:** anyone who asks "what should we eat?" every evening, solo, couples, and families.
- **The one job:** turn "I don't know" into a pick in under 10 seconds, with no stress.
- **Core loop:** open the app, tap Surprise me (or pick a mode), see one idea, then lock it in or spin again.

In v1

- Surprise me with spin-again, and four modes: Cook it, Fast food, Dine out, Delivery
- Mood filters (Comfy, Fresh, Spicy) and budget filters (`$` , `$$` , `$$$`)
- Faves and Past picks, with a 7-day no-repeat rule on Surprise me
- Real nearby restaurants and fast food for Dine out and Fast food, using the phone's location
- Simple recipes with a grocery list for Cook it

Not in v1

- Group voting ("Decide for us"), in-app ordering, calorie tracking, and paid plans. These come after launch.

Reference: the clickable prototype is the source of truth for look, copy, and behavior. Save its HTML file into the project at `reference/prototype.html` and have Claude Code open it before building any screen.

Look and feel

The app should feel like a playful kitchen party: bold colors, tilted shapes, big friendly type, and zero seriousness.

Token	Value	Use
Cream	`#FFF4DC`	App background
Ink	`#2B1B3D`	Text, borders
Tomato	`#FF4D3D`	"What" in the wordmark
Grape	`#7B4DFF`	"to" in the wordmark, Pick button
Mint	`#14B38A`	"eat?" in the wordmark, confirm button
Sunny	`#FFC93C`	Surprise me button, Fast food tile
Coral	`#FF8A7A`	Cook it tile
Lavender	`#B9A2FF`	Dine out tile
Seafoam	`#6FE0BF`	Delivery tile

- **Wordmark:** "What" tilted left at -7 degrees, "to" smaller and tilted right at 6 degrees, "eat?" largest at -3 degrees and nudged right. Keep it crooked on every screen.
- **Fonts:** Fredoka (600 and 700) for headings and buttons, Nunito (700) for body text.
- **Shapes:** thick 3px ink borders on main buttons, 18 to 28px rounded corners, tiles rotated 1 to 1.5 degrees in alternating directions.
- **Dark mode:** deep purple background (`#1D1230`) with the same bright accents, and cream text.
- **Motion:** a wobble on the result card while spinning, a small bounce on tap. Respect the reduce-motion setting.
- **Copy:** short, warm, sentence case. Examples: "Dinner, decided. No stress.", "Surprise me", "Spin again", "Let's eat this", "Start your shortlist".

Tech stack

Build one TypeScript codebase that ships to iPhone and Android, backed by a hosted database so there is no server to run.

Layer	Choice	Why
App	React Native with Expo (SDK current), TypeScript	One codebase for iOS and Android, fast to iterate, Claude Code knows it well
Navigation	Expo Router	File-based routes that match the screens list
Styling	NativeWind or plain StyleSheet with a `theme.ts` of the tokens above	Keeps colors and rotations in one place
Fonts	`@expo-google-fonts/fredoka`, `@expo-google-fonts/nunito`	Matches the prototype
Animation	`react-native-reanimated`	Wobble, spin, and tap bounce
Backend	Supabase (Postgres, Auth, Row Level Security, Realtime)	Accounts, faves, history, and later live group voting
Nearby places	Google Places API (New), called through a Supabase Edge Function	Keeps the API key off the phone and caches results
Recipes	Own `ideas` table seeded from the prototype, then expanded	Full control over the "30 minutes or less" rule
Location	`expo-location`	"Open now near you" for Dine out and Fast food
State and storage	TanStack Query for server data, `expo-secure-store` for session	Simple caching and offline-friendly
Delivery links	Deep links to DoorDash, Uber Eats, and Grubhub search pages	No ordering integration needed in v1
Builds and release	EAS Build and EAS Submit	TestFlight and Google Play internal testing
Analytics and errors	PostHog and Sentry	See which modes people use and catch crashes

Keys needed before building places: a Google Cloud project with Places API (New) enabled, and a Supabase project. Keep both keys in `.env`, never in the repo.

Screens and features

v1 has five screens plus a bottom tab bar (Home, Faves, Past picks), and each one already exists in the prototype.

Screen	What it does	Rules to keep
Home	Crooked wordmark, tagline, Surprise me button, four mode tiles	Surprise me picks from all modes and skips anything locked in during the last 7 days
Mode list	Mode title, mood chips, budget chips, "Pick one for me", scrollable list of ideas	Filters combine. If nothing matches, show "Nothing matches" and a hint to loosen filters
Result card	One idea in the mode's color: name, blurb, time, budget, mood, details, heart, "Let's eat this", "Spin again"	Spin again never repeats the current idea. Spinning takes about 1.3 seconds and shows a wobble
Faves	List of hearted ideas	Empty state: "Start your shortlist" with a button back to Home
Past picks	List of locked-in dinners with "Today", "Yesterday", or "N days ago"	Only "Let's eat this" writes to history. Browsing never does

Behavior that changes with real data in the full build

- **Cook it:** the result card opens a recipe view with ingredients, steps, and an "Add to grocery list" button.
- **Fast food and Dine out:** results come from Google Places near the user, filtered to open now, with distance, price level, and a "Get directions" button that opens the phone's maps app. Ask for location permission the first time, with a friendly explanation, and fall back to a city entry box if denied.
- **Delivery:** the result card has buttons that open DoorDash, Uber Eats, or Grubhub with the dish name pre-searched.
- **Accounts:** start in guest mode with data on the device. Offer sign-in with Apple and Google after the first fave so faves and history sync across phones.

After launch: "Decide for us" group voting, where two or more people swipe on shared options and the app picks the match.

Data model

Four tables cover v1. Ideas are shared and read-only, and everything else belongs to one user and is protected by Row Level Security.

Table	Key columns	Notes
`ideas`	id, mode, name, blurb, mood, cost, minutes, details, image_url	Seed with the 28 ideas in the prototype, then grow to 300 or more. Recipes add `ingredients` and `steps`
`favorites`	user_id, idea_id, created_at	One row per heart. Unique on user and idea
`picks`	id, user_id, idea_id, picked_at	Written only when the user taps "Let's eat this". Powers Past picks and the 7-day rule
`profiles`	user_id, display_name, home_city, created_at	Optional. Created at first sign-in

Restaurants and fast food come live from Google Places and are not stored, except a short cache of results keyed by place and time to cut API cost. Saved place favorites store the Google `place_id` and name.

```
create type mode as enum ('cook', 'fast', 'dine', 'deliv');
create type mood as enum ('comfy', 'fresh', 'spicy');

create table ideas (
  id bigint generated always as identity primary key,
  mode mode not null,
  name text not null,
  blurb text not null,
  mood mood not null,
  cost smallint not null check (cost between 1 and 3),
  minutes int,
  details text
);

create table favorites (
  user_id uuid references auth.users on delete cascade,
  idea_id bigint references ideas on delete cascade,
```

```
  created_at timestamptz default now(),
  primary key (user_id, idea_id)
);

create table picks (
  id bigint generated always as identity primary key,
  user_id uuid references auth.users on delete cascade,
  idea_id bigint references ideas,
  picked_at timestamptz default now()
);
```

For guest mode, keep the same shape in local storage and copy it into Supabase when the user signs in.

Build order

Build in six phases, and have Claude Code stop and show a working result at the end of each one before moving on.

1. **Foundation.** Create the Expo project, add the theme file, load the fonts, set up Expo Router with the three tabs, and build the crooked wordmark component. Done when the Home screen matches the prototype on a phone and in dark mode.
2. **Prototype port.** Rebuild all five screens with the 28 seed ideas in local state, including spin, filters, faves, and past picks saved on the device. Done when every tap in the prototype works the same way in the app.
3. **Backend.** Create the Supabase project, add the tables and Row Level Security, seed `ideas`, and swap local data for server data. Add guest mode, then Apple and Google sign-in. Done when faves and history follow the user to a second phone.
4. **Real places.** Add the Edge Function for Google Places, location permission, open-now filtering, distance, and directions for Fast food and Dine out. Done when a user in a new city sees real nearby results.
5. **Cook it and Delivery.** Add recipe view with ingredients and steps, the grocery list, and delivery deep links. Done when a user can go from Surprise me to a shopping list or an order page in three taps.
6. **Polish and ship.** Add animations, empty and error states, accessibility labels, analytics, and crash reporting. Build with EAS, test through TestFlight and Google Play internal testing, then submit. Done when 10 real testers pick a dinner without help.

Rules for Claude Code throughout

- Match the prototype's colors, copy, and behavior exactly unless told otherwise.
- Keep screens thin and put logic (picking, filtering, the 7-day rule) in plain functions with tests.
- Never put API keys in the app. Calls to Google go through the Edge Function.
- Commit after each phase with a short message saying what now works.

Start in Claude Code

Create an empty folder, put `reference/prototype.html` and this brief's `CLAUDE.md` (below) in it, open Claude Code there, and paste the kickoff prompt.

Kickoff prompt

```
I'm building a mobile app called "What to eat?" that helps people decide
dinner (cooking, fast food, dining out, delivery) with one fun tap. Read
CLAUDE.md first, then open reference/prototype.html: it is a working
clickable prototype and the source of truth for look, copy, and behavior.

Start with Phase 1 (Foundation) from CLAUDE.md: create an Expo + TypeScript +
Expo Router app, add a theme file with the color tokens, load Fredoka and
Nunito, build the crooked wordmark component, and set up the three tabs
(Home, Faves, Past picks). Make the Home screen match the prototype in light
and dark mode.

When Phase 1 runs, stop and tell me how to open it on my phone. Do not start
Phase 2 until I say so.
```

CLAUDE.md

```
# What to eat?

Mobile app that ends the nightly dinner debate: one fun tap gives one dinner
idea (Cook it, Fast food, Dine out, Delivery).

## Source of truth
reference/prototype.html is a working prototype. Match its colors, copy,
layout, and behavior unless told otherwise.

## Stack
Expo (React Native) + TypeScript, Expo Router, Reanimated, Supabase
(Postgres, Auth, RLS), Google Places API (New) via a Supabase Edge Function,
```

```
TanStack Query, EAS Build.

## Look
- Colors: cream #FFF4DC, ink #2B1B3D, tomato #FF4D3D, grape #7B4DFF, mint
#14B38A, sunny #FFC93C, coral #FF8A7A, lavender #B9A2FF, seafoam #6FE0BF.
Dark background #1D1230.
- Fonts: Fredoka for headings and buttons, Nunito for body.
- Wordmark is always crooked: What (-7 deg), to (6 deg, smaller), eat? (-3
deg, largest).
- Thick 3px ink borders on main buttons, rounded corners, tiles slightly
rotated.
- Copy is short, warm, sentence case.

## Rules
- Surprise me skips anything locked in during the last 7 days.
- Spin again never repeats the current idea.
- Only the Let's eat this button writes to Past picks.
- Keep logic (pick, filter, 7-day rule) in plain tested functions, not in
screens.
- Never put API keys in the app. Google calls go through the Edge Function.
- Respect reduce-motion and add accessibility labels on every control.

## Phases (stop and show a working result after each)
1. Foundation (project, theme, fonts, tabs, wordmark, Home)
2. Prototype port (all screens, 28 seed ideas, local storage)
3. Backend (Supabase tables, RLS, guest mode, Apple and Google sign-in)
4. Real places (Fast food and Dine out near the user)
5. Cook it recipes and grocery list, Delivery deep links
6. Polish, analytics, crash reporting, TestFlight and Play internal testing

## Commit
Commit after each phase with a short message saying what now works.
```